

CMPT 276 Assignment 4

Krish Kaushik

March 31, 2026

1. Code Smells Identified

I reviewed code implemented by **Tanveer Muntaseer** in `ObamaScreen.java` and `GameLogicHelper.java`, and identified the 4 code smells below.

Smell 1 - Dead Code `ObamaScreen.java`, `getProgressiveDialogue()`, lines 378–382

`totalDuration` is assigned `TALK_FALLBACK_DURATION` unconditionally on line 378, then the `if (SettingsScreen.soundOn && obamaVoice != null)` block immediately below assigns the exact same value again. The condition can never change the result, making the branch dead code that misleads future readers.

Suggestion: Delete the `if` block entirely.

Smell 2 - Inappropriate Intimacy `ObamaScreen.java`, lines 240, 251, 381

`ObamaScreen` directly accesses `SettingsScreen.soundOn` in three separate places. These two classes have no architectural relationship (one is a cutscene, the other is a settings UI) creating tight coupling that makes refactoring either class riskier.

Suggestion: Add a `private boolean soundOn` field to `ObamaScreen`, pass it as a constructor parameter, and replace all three direct references with the local field so the coupling lives in exactly one place.

Smell 3 - Long Method `ObamaScreen.java`, `render()`, lines 230–266

The TALK phase logic is handled entirely inline inside `render()`, while the other two phases already delegate to dedicated methods (`updateObamaWalk()`, `updatePlayerWalk()`). The inconsistency makes the method long and harder to read.

Suggestion: Extract the TALK case into a new `private updateTalkPhase(float delta)` method, so all three phases are handled the same way and `render()` becomes a clean coordinator.

Smell 4 - Data Clumps `GameLogicHelper.java`, lines 172–189

```
public static float getPlayerSpawnX(int mapIndex) { ... }  
public static float getPlayerSpawnY(int mapIndex, float[] mapYOffsets) { ... }
```

Two parallel switch methods represent one concept: a spawn position. Every call site must invoke both and pass `mapIndex` twice. Adding a new map requires updating both switches in sync; forgetting one produces a wrong position with no compiler warning.

Suggestion: Merge into a single `getPlayerSpawnPosition(int mapIndex, float[] mapYOffsets)` returning a `Vector2`, so each map's coordinates are defined in one place.

2: Refactoring My Own Code

After receiving a review from Samantha Gan, I addressed four code smells she identified in `PoliceEnemy.java` and `GameScreen.java`. Each fix was committed and pushed separately; `mvn test` was rerun after each commit and all tests passed every time.

Fix 1 - Duplicated Code `PoliceEnemy.java`

commit 61d29a3

The barrier-collision logic – computing `hitW`, `hitTop`, and four `isBarrier()` checks for both axes – was copy-pasted verbatim in both the chasing branch and the wander branch of `update()`. I extracted it into a private

`applyBarrierCollision(float newX, float newY, BarrierLookup lookup)` method (line 189) and replaced both duplicated blocks with a single call.

Fix 2 - Long Method: `update()` `PoliceEnemy.java`

commit 2ccdb8a

`update()` handled the alert state machine, movement, and animation all inline across roughly 125 lines. I extracted three private helpers – `updateAlertState`, `updateMovement`, and `updateAnimation` – each with a single responsibility, reducing `update()` to four lines:

```
float dist = Vector2.dst(position.x, position.y, playerX, playerY);
updateAlertState(delta, dist);
updateMovement(delta, playerX, playerY, lookup);
updateAnimation(delta);
```

Fix 3 - Dead Code `GameScreen.java`

commit cae4571

`activateMap()` called `police.clear()` then immediately checked `police.size() < 3` and `police.size() < 16` which is always true after a clear, making both guards dead code. When I implemented the clearing, I never changed the police size checks. To address this, I removed them and I also extracted the 33 hard-coded spawn calls into named constant 2D arrays (`MAP2_POLICE_SPAWNS`, `MAP3_POLICE_SPAWNS`) and a private `spawnEnemiesForMap(int index)` helper, so `activateMap()` delegates to a single call and spawn coordinates are defined once at the top of the class.

Fix 4 - Low Cohesion: `render()` `PoliceEnemy.java`

commit 8711311

`render()` was doing two unrelated things: drawing the police sprite and drawing the alert indicator (? or !) overlay. I split it into two private methods: `renderSprite(batch)` and `renderAlertIndicator(batch)`. Now each has exactly one reason to change, and `render()` simply calls both.